Day 14 Report: AI Agent from Scratch

Tool calling and the ReAct loop (Reason, Act, Observe), built without an agent framework

1. Goal

Build an **AI agent**: an LLM that can **use tools**. It reasons about the goal, picks a tool, looks at the result, and repeats until it can answer. The loop was written by hand (no LangChain or other framework), so every step is visible.

2. How it works

ReAct loop

Tool	What it does	Times used
calculator	Safe maths (only numbers and maths functions, no other code can run)	11
search_papers	Day 13 RAG index over 5 AI papers (FAISS + re-ranking)	6
web_search	DuckDuckGo, with Wikipedia as a backup	5
get_today	Returns today's date	1

Each tool has a **JSON schema** (name, description, arguments). The model chooses a tool only from these descriptions. Tool errors are returned to the model as text instead of crashing the program, and **MAX_STEPS = 8** stops endless loops.

Tool-calling mode used in this run: *prompt-based*. No Gemini API key was set, so Colab's built-in Gemini was used and it wrote each tool call as a JSON object, which my code parsed and ran. The notebook also contains the **native** version (Gemini function calling); it runs automatically when a `GEMINI_API_KEY` is added to Colab Secrets.

3. Results (10 multi-step questions)

Metric	Result
Success rate (automatic check)	90% (9 of 10)
Average model calls per question	3.3
Average tool calls per question	2.3
Average time per question	19.8 s
Average tokens per question (estimated)	3,375
Questions that hit the 8-step limit	0

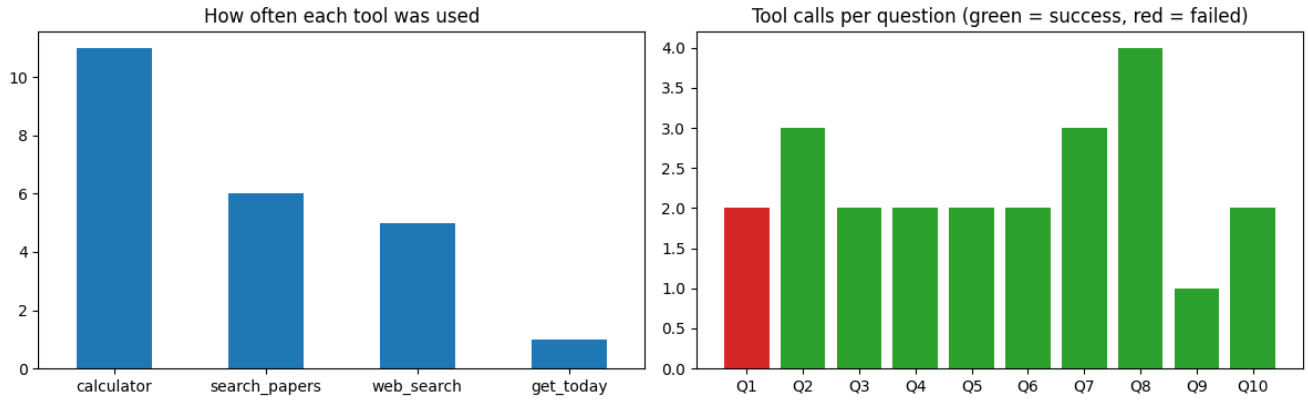

Figure 1. Tool usage, and tool calls per question (green = success, red = failed)

4. Example trace (Q7)

Step	What the agent did
1 Think / Act	"Find the first author and year of LoRA" -> search_papers(...)
1 Observe	Paper passages did not show the author list
2 Think / Act	"The papers didn't list the authors, use the web" -> web_search(...)
3 Think / Act	"Edward Hu, 2021; now compute 2030 - 2021" -> calculator('2030 - 2021') = 9
4 Answer	"The first author is Edward Hu, published in 2021, 9 years before 2030."

This shows real agent behaviour: when one tool did not help, it **changed strategy** and tried another.

5. Failure analysis

Question	What happened	Type of problem
Q1 Tokyo population / 1,000	Used 10,316,210 from a Wikipedia demographics page and answered 10,316. Current figure is about 14 million (city) or 37 million (metro).	Tool returned old / different data; agent did not check the source
Q8 Transformer heads	Called calculator with no argument, got an error, then retried correctly (512 / 8 = 64).	Recovered: error handling worked
Q10 Boiling point	Right answer (373.15 K), but it calculated 100 + 273.15 instead of converting from 212 F.	Shortcut: right answer, weaker reasoning
Q9 Days to 4 Feb	Used '31 - 1 + 4' = 34 (correct), no date tool needed.	OK

6. Key findings

- The agent solved **9 of 10** multi-step questions and never hit the step limit.
- It used the **calculator for all maths** instead of guessing numbers, as instructed.
- It picked the right tool most of the time: **papers** for paper questions, **web** for world facts.
- **Returning errors as text** lets the agent fix its own mistakes (Q8).
- The weakest point is **trusting search results** without checking how recent or reliable they are (Q1).

7. Limitations and next steps

- Run again with a Gemini API key to test **native** tool calling and get exact token counts.
- Ask the agent to prefer the latest figures and mention the source and year of a number.
- Only 10 test questions; the automatic check is simple, so traces were also read by hand.
- Day 15 turns this agent into a reusable **harness** (tool registry, memory, guardrails, tracing).